ADR 0132 — A Settings read never negotiates with a locked keyring

- **Status:** Accepted — implemented, mutation-proved two ways failing at disjoint assertions
- **Date:** 2026-09-06
- **Issue:** #692
- **Extends:** ADR 0126 (#583), whose resolver precedence remains the credential-operation contract
- **Supersedes:** ADR 0129 decision 3 only — live keyring resolution on every Settings request

Context

`resolve_token` claimed to stop at the first source with a value, but passed four already-evaluated `Option<String>` arguments to `resolve_from`:

```
resolve_from(
    keyring_token(),
    env_token(GIT_VISTA_ENV),
    env_token(GH_ENV),
    file_token(),
)
```

Rust evaluates those arguments before entering `resolve_from`. Every call therefore read Secret Service, both environment variables, and the fallback file even when the first tier won. The returned precedence was right; the observable evaluation contract was not.

ADR 0129 made the more important part request-facing. Every Settings GET, and the read-back after every successful POST, called `resolve_token` synchronously inside an async handler. On Linux, keyring v1's configured backend talks to Secret Service over D-Bus and may ask it to unlock matching items. That is neither a reliably fast local computation nor async I/O.

The concrete failure host is an autologin desktop with no unlockable login keyring. Opening Settings can begin an unlock interaction that cannot complete, occupy a Tokio runtime worker, and repeat on every open. This is a local UX/availability defect, not credential disclosure: the route remains session-gated, full-listener-only, and the response remains masked.

Two contracts must not be conflated:

- a credentialed git operation needs the actual current token, so its live resolver may have to read the highest-precedence store; and
- Settings needs a bounded, masked statement of configuration. It does not need the raw keyring value on every request.

Decision

1. Resolution receives source functions and stops invoking them at the winner

`resolve_from` accepts four `FnOnce() -> Option<String>` sources in the documented order:

1. OS keyring;
2. `GIT_VISTA_GITHUB_TOKEN`;
3. `GH_TOKEN`; and
4. the fallback file.

It calls one source at a time and returns immediately when one answers. Tests inject functions with independent counters for every possible winning tier. They assert both provenance and the exact call vector (`[1,0,0,0]`, `[1,1,0,0]`, and so on), rather than testing only pre-built values.

This changes evaluation, not precedence or provenance. The ordinary `resolve_token` used by credentialed operations still returns the raw token plus the same `TokenSource`.

2. Startup makes the sole Settings keyring read, off the runtime worker and under a two-second budget

Before either listener is built, `initialize_request_token_policy` launches one `keyring_token` call with `tokio::task::spawn_blocking` and waits at most two seconds. The result seeds a process-lifetime `RequestTokenResolver` shared by the loopback and LAN router builds (the LAN router still does not register the Settings routes).

If the probe finds a token, the resolver retains only a `TokenStatus` holding its fixed source label and masked tail. It does not retain another raw-token copy. If the probe returns no token, fails, or exceeds the budget, no keyring status is retained. Timeout/worker failure also emits a startup warning that the keyring is unavailable to Settings until restart.

A synchronous call already running on Tokio's blocking pool cannot be cancelled by dropping its `JoinHandle`. A timed-out probe may therefore leave one blocking task behind until D-Bus answers. The bound is on server startup and every Settings request, not on that platform call's lifetime. Crucially, another dialog open does not create another task or unlock interaction.

3. GET reads the masked snapshot, then only the live non-keyring tiers

`GET /api/settings/token` has no keyring source function. It follows this policy:

1. If startup or a successful POST established a masked keyring snapshot, return it immediately. Lower tiers cannot run.
2. Otherwise evaluate `GIT_VISTA_GITHUB_TOKEN`, `GH_TOKEN`, and the fallback file lazily and return the first winner, masked.
3. If none answers, return `configured: false`.

The environment and file tiers remain live because they do not negotiate an unlock. The keyring observation is deliberately process-lifetime state. If an external program adds, removes, unlocks, or changes the entry after startup, Settings will not observe that change until a successful Settings POST or a server restart.

That staleness is explicit and narrower than caching the credential resolver: credentialed git operations still call live `resolve_token`, and the cache contains only masked display state. Settings now answers the bounded question "what configuration did this process establish for status?", not "can Secret Service prove the same fact again right now?"

4. POST performs one explicit write on the blocking pool and never reads back

`POST /api/settings/token` is different from GET because the operator asked to access the keyring. Its synchronous `set_password` runs inside `spawn_blocking`, so no Tokio runtime worker is occupied by D-Bus.

On success, the same blocking task updates `RequestTokenResolver` from the normalized submitted value and returns the resulting masked `TokenStatus`. There is no `get_password` read-back. Updating beside the write also means a client disconnect after submission cannot leave a completed write and an unchanged snapshot merely because the handler future was dropped.

On blank input or keyring failure, the existing `400`/`502` response mapping remains and the snapshot is unchanged. POST has no artificial timeout: a synchronous write cannot be cancelled safely, and reporting a timeout while the detached call later succeeds would turn a definite write API into an ambiguous one. The blocking-pool boundary protects runtime availability; the explicit save may still wait for the platform credential service it asked to use.

5. The clone-containment boundary does not move

This decision does not alter ADR 0128's credentialed-transfer, credentialless-checkout, environment-clearing, or output-redaction phases. The only shared helper change is `resolve_from` becoming lazy. The live credential resolver and its returned token/provenance shape are otherwise unchanged.

Exact behavior on the locked autologin host

At server start, one status-related keyring probe runs off the Tokio worker. If it cannot finish within two seconds, startup continues, warns that Settings will treat the keyring as unavailable until restart, and evaluates the environment/file fallbacks for the startup provenance line.

Every later Settings GET answers without D-Bus. It reports the first live environment/file fallback, or "not configured" when those are absent. It does not reopen an unlock dialog, add another blocked task, or occupy a runtime worker. A successful explicit Save replaces that status with the masked keyring result; otherwise restart is the retry boundary.

Credentialed clone/push/fetch behavior is not silently changed by that request policy: an operation needing the real credential still uses the live resolver and may encounter the platform keyring independently.

Alternatives considered

Run the full resolver in `spawn_blocking` on every GET. Rejected. It would protect Tokio's runtime workers but preserve the unusable interaction and create another potentially stuck blocking task every time Settings opens.

Wrap every GET lookup in a timeout. Rejected for the same reason. Timeout does not cancel synchronous D-Bus work; repeated requests

would bound their own response times while leaking repeated background work and prompts.

Cache the raw keyring token for the whole process and use it for both Settings and credentialed operations. Rejected. Status needs only a masked fact, and lengthening the lifetime of an extra raw credential copy is not necessary to fix the request path. It would also silently turn the live credential resolver into a startup snapshot.

Never inspect the keyring for status. Rejected. A bounded startup observation preserves useful provenance for the ordinary healthy-keyring case without putting that negotiation on a user-triggerable request.

Time out POST just like the startup read. Rejected. Once `set_password` has begun it cannot be cancelled; returning failure while it can still succeed later is an ambiguous mutation contract. The explicit write is instead moved wholly to the blocking pool and awaited.

Consequences

- Healthy keyrings are inspected once for Settings per process, not once per dialog open. Only masked status survives that observation.
- Locked/unavailable keyrings require a successful Settings POST or restart before Settings will reconsider the tier.
- Environment/file changes remain visible to GET whenever no keyring snapshot wins.
- The startup provenance line and request snapshot share the one bounded keyring observation, avoiding an immediate second D-Bus call.
- Status can be stale relative to an external keyring mutation; this is the accepted availability trade-off and is no longer documented as a live credential-store health check.
- The ordinary credential resolver remains live for credentialed operations, but now avoids every lower-priority source after a winner.

Verification

- `resolution_evaluates_source_functions_only_through_the_winning_tier` injects four counters and proves each exact evaluation boundary.
- `repeated_request_status_calls_never_repeat_the_startup_keyring_probe` proves three status reads leave the one probe count unchanged.

- `a_stuck_startup_keyring_probe_times_out` drives a blocked injected source through the real timeout wrapper.
- `request_status_uses_the_masked_keyring_snapshot_without_lower_reads` proves a keyring snapshot prevents every lower-tier function from running.
- `keyring_store_work_runs_off_the_async_runtime_thread` records both thread identities under a current-thread Tokio runtime and proves they differ.
- `a_successful_store_updates_status_without_a_keyring_read_back` proves the returned/persisted display state is the masked keyring tier.

Mutation proof

Failure Atlas ran both experiments from clean commit `ce464980` with green baselines and no warnings:

1. `mutation_check` **#369** inserted a `file()` call below a winning keyring source. The lazy-boundary test failed on the exact extra evaluation: `[1, 0, 0, 1]` observed against `[1, 0, 0, 0]` expected.
2. `mutation_check` **#370** removed the successful-write assignment to the masked keyring snapshot. The post-write test failed at a disjoint status assertion: unconfigured/empty observed against configured/`...tail` from `OS keyring` expected.

Both outcomes were `caught`; one proves the general resolver's evaluation boundary, while the other proves request status is advanced after an explicit write without a D-Bus read-back.